Project Report: Chessboard Detection and Piece Classification

Leonardo Gusson Vincenzo Sammartino

February 21, 2026

1 Project Introduction and Objectives

The project aims to develop a computer vision system capable of automatically detecting the positions of chess pieces on an 8x8 board by analyzing images captured from multiple perspectives. The primary goal of the developed system is to produce a detailed digital representation of the current board state. Given one or more images of a physical board as input, the processing pipeline localizes the various elements and identifies their specific types (i.e., pawn, knight, bishop, rook, queen, and king). This is achieved by employing a hybrid approach: while chessboard detection relies entirely on traditional computer vision techniques, piece classification combines a Bag of Visual Words (BoW) approach based on SIFT features with a Random Forest machine learning model. Although the initial project proposal included the ability to distinguish the color of the pieces, our current implementation primarily focuses on spatial localization and piece class recognition.

2 Implementation strategy

The execution flow has been divided into three main procedures, sequentially activated by the `main.cpp` program.

First, the ML model is trained using the provided dataset (18,141 training images). This training step is optional if the user decides to skip it and the system detects a `.yaml` file containing a previously trained model. During training, it computes SIFT features to build a matrix of labeled descriptors, which are then clustered using the K-Means algorithm to generate a vocabulary of 300 centroids.

Then, based on user input, the pipeline processes either a single test image or the entire test set (55 images). It applies computer vision algorithms such as line detection and various filters to detect the chessboard and extract its individual squares. An initial filtering step is performed to identify empty and potentially occupied squares.

Finally, the non-empty squares are passed to the chess classifier to test the extracted patches against the trained model, ultimately computing performance metrics to evaluate the overall quality of the detection and classification.

3 Code Structure and Workload Distribution

The project's source code is structurally organized into two primary directories: `include/` containing the header declarations and `src/` containing the core implementation files.

The software architecture enforces a strict separation of concerns, clearly dividing the chessboard and piece detection pipeline from the piece classification and evaluation metrics. This modularity directly reflects the distinct contributions of the authors. Specifically:

- **Leonardo Gusson** (60h) designed an initial version of the system architecture, creating the first drafts of the interfaces and class skeletons. Then worked on implementing the methods related to input/output operations (`io_functions.cpp`), the geometry helper functions and the computer vision filters applied to the test images (`geometry_utils.cpp` and `board_detector.cpp`) to perform the chessboard detection;
- **Vincenzo Sammartino** (60h) developed the complete chess classification module (`chess_classifier.cpp`), implementing the training and inference pipeline. He designed the preprocessing strategy (grayscale conversion, optional Gaussian smoothing, conditional CLAHE), implemented descriptor filtering and histogram normalization, managed class balancing during training, and integrated vote-based confidence estimation at inference time. Then implemented the quantitative evaluation framework (`evaluation.cpp`).

The project exclusively relies on the OpenCV library, completely avoiding deep learning frameworks. Specifically, we employed the `core`, `highgui`, `imgproc`, `imgcodecs`, `features2d` and `ml` components. Here is the correct code structure of the repository.

```
CV_FinalProject
├── include/
│   ├── board_detector.h
│   ├── chess_classifier.h
│   ├── chessboard.h
│   ├── classifier_eval_config.h
│   ├── evaluation.h
│   ├── geometry_config.h
│   ├── geometry_utils.h
│   └── io_functions.h
```

```
CV_FinalProject
├── src/
│   ├── board_detector.cpp
│   ├── chess_classifier.cpp
│   ├── chessboard.cpp
│   ├── evaluation.cpp
│   ├── geometry_utils.cpp
│   ├── io_functions.cpp
│   └── main.cpp
└── CMakeLists.txt
```

4 Dataset

The main dataset was acquired directly from the link provided in the project proposal. It is available here through the Roboflow platform. It consists of 19,708 images, each with a resolution of 416x416 pixels. Although the dataset provides annotations in various formats, we specifically utilized the YOLOv5 annotations (`.txt` files). Due to the low resolution and high noise of the training images, we attempted to improve performance by building an additional dataset, merging chess pieces images from various online sources. However, this supplementary dataset was used exclusively for training purposes. The dataset folder organization within our repository is outlined below.

```
CV_FinalProject
├── input_files
│   ├── dataset1
│   │   ├── train
│   │   ├── valid
│   │   └── test
│   ├── dataset2
│   │   └── train
│   │       └── ...
│   ├── bishop.png
│   ├── king.png
│   ├── knight.png
│   ├── pawn.png
│   ├── queen.png
│   └── rook.png
```

```
CV_FinalProject
└── output_files
    ├── debug_images
    │   └── ...
    ├── chess_rf.yml
    ├── chess_vocabulary.yml
    ├── chessboard_2d_0.png
    ├── chessboard_2d_1.png
    └── ...
```



Figure 1: Training set sample image

The C++ code is made aware of the repository folder structure through CMake. Instead of relying on hardcoded absolute paths, the build system injects specific compilation constants (`INPUT_FILES_DIR`, `OUTPUT_FILES_DIR`, `DATASET1_DIR`, and `DATASET2_DIR`) directly into the code.

5 Accessing the Repository

The complete repository (source code and datasets) is available on GitHub here. Alternatively, the source code can be used in conjunction with the `input_files` and `output_files` folders, which can be downloaded from the Google Drive folders here and here. If downloaded manually, these folders must be placed on the project's root, as described above.

6 Board Detection

The primary objective of the board detection module is to identify the chessboard structure and isolate the individual squares present within the input image. This core functionality is entirely encapsulated within the `BoardDetector` class and is orchestrated by its main public method, `detect`. By taking a raw BGR test image as input, this method coordinates a sequence of computer vision operations designed to locate line segments, reconstruct the mathematical grid, and evaluate the occupancy of each resulting cell. The underlying processing pipeline is structured into several progressive phases, which are explored in detail in the following sections.

6.1 Segment Extraction (LSD)

The first step in the board detection pipeline involves extracting raw line segments from the grayscale image. While an initial preprocessing stage utilizing Gaussian Blur and/or CLAHE was evaluated to reduce noise and enhance contrast, it was ultimately discarded due to the inherently low resolution of the input images. Empirical testing demonstrated that the best performance is achieved by feeding the raw grayscale image directly into OpenCV's Line Segment Detector (LSD). Since the LSD algorithm typically detects many micro-segments due to image noise or texture, we applied an initial length-based filter. Specifically, any segment shorter than a predefined percentage of the image's maximum dimension is discarded (blue segments in figure 2a).

Following this preliminary filtering, the remaining segments are merged to group collinear lines into larger structural clusters. The algorithm processes each segment by computing its corresponding mathematical line equation and direction vector. It then evaluates the segment against previously established clusters: if the new segment is parallel to a cluster's reference line (within a strict angular threshold) and sufficiently close to it (based on a perpendicular distance threshold), it is added to that group, and the group's reference line is updated as an incremental average. Otherwise, it initializes a new cluster.

Finally, all segments belonging to the same cluster are projected onto their shared 1D reference line, preparing the data for subsequent geometric analysis.

6.2 Geometric, Angular, and Spatial Filtering

Following the formation of initial clusters, a geometric filtering stage evaluates the quality of each cluster based on its 1D projection.

Clusters having poor overall line coverage, unexpected large gaps between member segments or excessive coverage indicative of image noise (e.g., covering over 95% of the image's maximum dimension) are systematically discarded (figure 2b). Next, to isolate the grid's primary orthogonal axes, the remaining clusters undergo angular K-Means ($K = 2$) clustering. The algorithm maps the doubled angles of the reference lines to a 2D point, $(\cos(2\theta), \sin(2\theta))$, to handle the $0 - \pi$ angular discontinuity. Outliers deviating significantly from the computed centroids are here rejected.

Subsequently, an angular filter analyzes the variance within each directional group. For low-variance (nearly parallel) configurations, it directly removes lines deviating from the angular mean. On the other hand, for high-variance groups, typically resulting from perspective distortion, it evaluates pairwise line intersections to find the best vanishing point, retaining only the converging inlier lines (figure 2c).

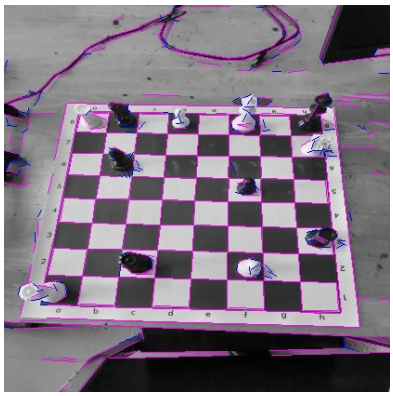
Finally, a spatial filtering step exploits the periodic structure characteristic of a chessboard. The horizontal and vertical lines are sorted spatially based on their intersection with the image's central axes. By analyzing the distance between consecutive lines, the algorithm identifies "seeds"—subsets of consistent gaps that reliably estimate the average cell dimension. These seed regions are iteratively expanded outward, guided by a strict expansion tolerance, to reconstruct the complete grid framework (figure 2d).

6.3 Grid Extraction and Occupancy Analysis

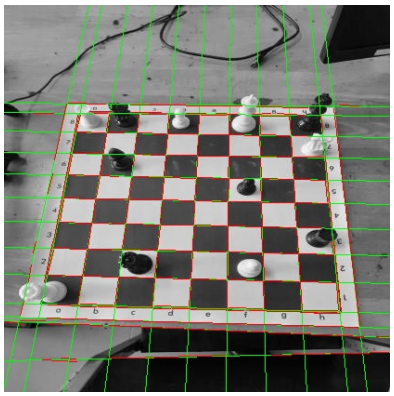
After the isolation of the valid grid lines, the algorithm computes the 2D intersection points for every pair of orthogonal lines (horizontal and vertical). By grouping the intersections of adjacent lines, it defines the four corners of each individual cell composing the board.

Once the spatial boundaries of the cells are available, a uniformity check is performed to determine piece occupancy. The algorithm evaluates each cell using Canny edge detection alongside the standard deviation of its internal grayscale intensities. To prevent false positives caused by adjacent pieces spilling over the cell borders, the analysis is restricted to a morphologically shrunk mask of the polygon. A cell is classified as empty only if it exhibits a low standard deviation in pixel intensity and a small number of internal edge pixels (see figure 2e).

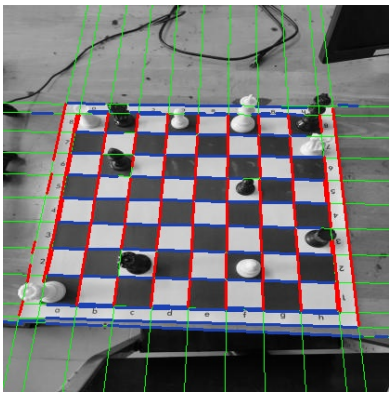
Finally, to establish the correct orientation of the board, the algorithm estimates the top-left cell color. A global average grayscale threshold is computed across the entire detected board area to differentiate between light and dark squares. The algorithm then randomly samples up to five distinct empty cells. For each sampled



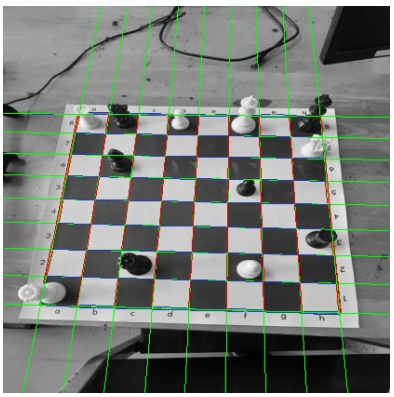
(a) LSD output



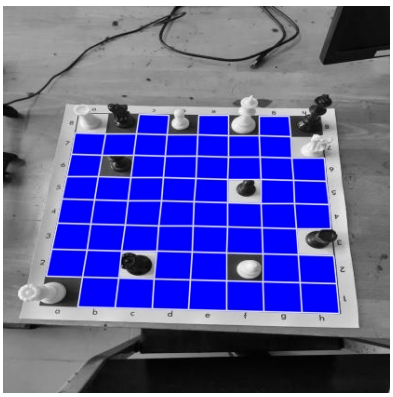
(b) Geometric filter output



(c) Angular filter output



(d) Spatial filter output



(e) Grid view, with empty cells highlighted

Figure 2

cell, its mean intensity is compared against the threshold to classify it as light or dark. Exploiting the geometric pattern of a chessboard, each evaluated cell casts a vote for the color of the origin square. The final top-left color is determined by majority vote.

7 Chess Classification

The classification module is responsible for assigning a semantic label (BISHOP, KING, KNIGHT, PAWN, QUEEN, ROOK) to each geometrically detected chessboard cell. While the detection stage operates at full-image level and focuses on geometric reconstruction of the board, the classification stage works at patch level and performs purely semantic discrimination.

The classifier is implemented inside the `ChessClassifier` class and independently processes image patches corresponding to individual board cells. Only non-empty cells are forwarded to the classifier. This design choice ensures modularity and allows the classification stage to remain fully decoupled from geometric reasoning, which significantly reduces intra-class variability and stabilizes descriptor extraction.

The overall classification pipeline consists of four sequential steps: image preprocessing, local feature extraction using SIFT, Bag-of-Visual-Words encoding, and multi-class Random Forest inference. The same processing chain is applied consistently during both training and testing, ensuring statistical alignment between the two distributions.

7.1 Image Preprocessing

A unified preprocessing routine is applied consistently during both training and inference prior to SIFT extraction. The goal of this stage is to stabilize local gradient statistics while preserving the fine structural details of chess pieces.

Each input patch is first converted from BGR to grayscale, as SIFT operates on single-channel intensity images. Gaussian smoothing is applied moderately to avoid blurring discriminative contour details.

To improve robustness on low-contrast samples, an adaptive contrast enhancement strategy is adopted. For each patch, the grayscale standard deviation is computed; if it falls below a fixed threshold ($\sigma < 25$), Contrast Limited Adaptive Histogram Equalization (CLAHE) is applied using a clip limit of 2.0 and a tile grid size of 8×8. This conditional enhancement increases local contrast in poorly illuminated regions while preserving edge structures, thereby improving descriptor stability without introducing artificial gradients.

7.1.1 Preprocessing of Training Images

The training data consists of annotated chessboard images (Dataset1) and additional single-piece images (Dataset2). For Dataset1, object annotations provided in YOLO format are converted into pixel coordinates and used to crop the corresponding Regions of Interest (ROIs), thereby isolating each chess piece from the surrounding board context. The binary masks associated with each object are used to filter out descriptors belonging to background regions. Only keypoints falling inside the object mask are retained, ensuring that vocabulary construction is driven exclusively by piece-related visual patterns. This reduces contamination from board textures and improves the semantic purity of the visual words.

Scale invariance and limited rotation robustness are handled intrinsically by the multi-scale SIFT detector.

7.1.2 Design Considerations

The preprocessing strategy is intentionally lightweight. The objective is not to impose strong normalization constraints, but rather to stabilize local intensity distributions while preserving fine-grained contour information. Geometric alignment is guaranteed by the detection stage, whereas scale and rotation tolerance are provided by SIFT itself.

By applying an identical preprocessing pipeline in both training and inference, the classifier operates on statistically consistent inputs, reducing distribution shifts that could otherwise degrade performance.

7.2 Feature Extraction and Bag of Visual Words (BoW)

After preprocessing, each image patch is represented through a set of local invariant descriptors extracted using SIFT. Since the number of detected keypoints varies across patches, a fixed-length representation is required in order to train a standard classifier. For this reason, a Bag-of-Visual-Words (BoW) model is adopted.

7.2.1 SIFT Feature Extraction

For each grayscale patch, SIFT detects keypoints in scale-space using the Difference-of-Gaussians operator and computes 128-dimensional descriptors based on local gradient orientation histograms. These descriptors encode local edge and contour structures that are particularly informative for distinguishing between chess-piece silhouettes.

To control computational cost and avoid instability in small patches, the number of extracted descriptors per patch is bounded. This prevents dominant regions from overwhelming the histogram representation and stabilizes the subsequent quantization stage.

7.2.2 Vocabulary Construction via Global K-Means

All descriptors extracted from the training ROIs are aggregated and clustered using K-Means. The clustering objective minimizes the within-cluster variance:

$$\min_{\{\mathbf{c}_k\}} \sum_i \min_k \|\mathbf{d}_i - \mathbf{c}_k\|_2^2$$

where $\mathbf{c}_k$ are the cluster centroids.

The resulting centroids define the visual vocabulary, with vocabulary size fixed to

$$K = 300$$

.

To limit memory consumption and clustering time, the total number of descriptors used for vocabulary construction is capped (maximum 300k descriptors in the final configuration). This subsampling preserves descriptor diversity while maintaining computational feasibility.

7.2.3 Descriptor Quantization and Robust Assignment

Given a patch, each descriptor is assigned to its nearest visual word using Euclidean distance (hard assignment):

$$k^*(i) = \arg\min_k \|\mathbf{d}_i - \mathbf{c}_k\|_2.$$

However, raw nearest-neighbor assignment may include unstable or noisy descriptors. To improve robustness, the distribution of assignment distances is analyzed for each patch.

Let $d_i = \|\mathbf{d}_i - \mathbf{c}_{k^*(i)}\|_2$ be the matching distance. The median distance med is computed, and only descriptors satisfying

$$d_i \leq 1.5 \cdot \text{med}$$

are retained.

This filtering removes descriptors that are poorly represented by the learned vocabulary, reducing histogram noise and improving class separability.

7.2.4 Histogram Encoding and Normalization

The retained assignments are accumulated into a histogram:

$$h_k = \sum_i \mathbf{1}[k^*(i) = k]$$

where $\mathbf{1}[\cdot]$ is the indicator function.

The histogram is L1-normalized:

$$\hat{h}_k = \frac{h_k}{\sum_{j=1}^{K} h_j}.$$

L1 normalization makes the representation invariant to the number of detected keypoints and converts raw counts into empirical word frequencies. This is particularly important because patch texture density varies significantly between different piece types (e.g., pawns vs queens).

The normalized histogram $\hat{\mathbf{h}} \in R^K$ constitutes the final fixed-length feature vector used for Random Forest training and inference.

7.3 Model Training and Inference (Random Forest)

The final classification stage is implemented using a multi-class Random Forest trained on the fixed-length BoW histograms extracted from the training ROIs. The model is built using OpenCV `cv::ml::RTrees`, which provides a non-linear ensemble classifier based on decision tree aggregation.

Each training sample is represented by a normalized histogram $\hat{\mathbf{h}} \in R^K$, where $K = 300$ is the vocabulary size. The output space consists of six semantic categories.

The label `NONE` (empty square) is not part of the classifier output space, since empty-cell rejection is handled upstream during the occupancy analysis stage. This design reduces ambiguity in the decision boundary and allows the forest to focus exclusively on discriminating between piece types.

The Random Forest is trained in `ROW_SAMPLE` mode. Trees are grown with a maximum depth of 16, a minimum sample count of 2 per node, and a maximum of 200 trees. These parameters were chosen to balance expressive capacity and overfitting control, considering the relatively compact feature dimension.

Let T denote the number of trees in the ensemble. During inference, each tree independently produces a class prediction. If v_c denotes the number of trees voting for class c, the predicted label is obtained as:

$$\hat{y} = \arg\max_c v_c.$$

The confidence score associated with the prediction is computed as the normalized vote fraction:

$$\text{conf} = \frac{\max_c v_c}{T}.$$

This quantity represents the empirical agreement among trees and can be interpreted as a measure of decision stability. High confidence indicates consistent voting across the ensemble, whereas low confidence reflects ambiguity in the feature space.

To mitigate class imbalance across piece categories, a balancing procedure is applied before training. Let N_c be the number of training samples for class c. The effective training size per class is defined as:

$$N_{\text{target}} = \min_c N_c.$$

Each class is randomly subsampled to N_{target} examples, ensuring a uniform class distribution. This prevents dominant categories (e.g., pawns) from biasing tree splits during training and improves generalization across visually less frequent pieces.

8 Evaluation and Metrics

This section describes the quantitative evaluation protocol adopted to assess both detection and classification performance on the complete test dataset.

8.1 Evaluation Methodology

Performance evaluation is conducted by comparing predicted detections against ground-truth annotations provided in YOLO format. For each test image, YOLO labels are converted into pixel-based bounding boxes, which serve as reference objects.

Predicted detections are generated by the board detection stage, which reconstructs the 8×8 grid and extracts occupied cells, followed by the classification stage, which assigns a semantic label to each detected piece. To establish correspondence between predictions and ground truth, a matching procedure is applied. A predicted bounding box B_p and a ground-truth box B_{gt} are considered compatible if two conditions are satisfied:

1. The Euclidean distance between their centers is below a predefined threshold, ensuring geometric proximity;
2. Their Intersection over Union (IoU) exceeds a minimum threshold, defined as

$$\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|}.$$

Each ground-truth object can be matched to at most one prediction. A prediction satisfying the IoU constraint with a unique ground-truth box is counted as a True Positive (TP). Unmatched predictions are counted as False Positives (FP), while unmatched ground-truth objects are counted as False Negatives (FN).

Detection performance is summarized through Precision, Recall, and F1-score:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Classification accuracy is computed exclusively on IoU-matched detections, therefore measuring semantic correctness only when geometric localization is valid. This separation allows independent assessment of spatial and semantic performance.

9 Results

The evaluation was conducted on the complete test set, comprising 55 images.

9.1 Board and pieces detection

The `BoardDetector` module correctly processes 51 chessboard images; however, in 4 instances, an outer line of cells is missing.

On average, each image contains 20.78 ground-truth objects, while the system generates 20.40 predictions. Of these, an average of 18.29 are successfully matched with ground-truth boxes based on the IoU criterion.

Detection performance reaches:

$$\textbf{Mean Detection F1-score} = 87.84\%.$$

This result indicates that the geometric pipeline is highly reliable. The grid reconstruction and occupied-cell extraction stages produce spatially accurate detections with limited over-segmentation or missed objects.

9.2 Classification

When restricting the evaluation to geometrically valid matches, classification performance achieves:

$$\textbf{Mean Classification Accuracy (on matches)} = 28.94\%.$$

The mean confidence score computed from ensemble vote fractions is:

$$\textbf{Mean Confidence} = 54.52\%.$$

On average, approximately 5.81 pieces per image are correctly classified among the IoU-matched detections.

The significant discrepancy between detection and classification performance reflects the intrinsic limitations of the adopted representation. While the geometric stage relies on strong structural constraints imposed by the chessboard layout, the classification stage depends solely on local descriptor statistics encoded through the Bag-of-Visual-Words model. In a fine-grained setting such as chess-piece recognition—where classes share similar local gradient patterns and differ primarily in global shape— the absence of explicit spatial modeling reduces class separability. The moderate confidence score further suggests that ensemble agreement is often limited, indicating overlapping distributions in the BoW feature space.

Overall, the evaluation highlights a structurally robust detection framework paired with a computationally efficient yet semantically constrained classification module. The observed performance gap is therefore coherent with the deliberate choice of adopting lightweight, classical computer vision techniques.

10 Conclusions

This project presented a complete computer vision pipeline for automatic chessboard analysis, including geometric board detection and semantic piece classification. The system was entirely developed using classical computer vision techniques and ML in C++ with OpenCV, without relying on deep learning models.

The detection stage demonstrated strong robustness. Through edge extraction, line detection and grid reconstruction, the system reliably localized the 8×8 board structure and identified occupied cells. The high mean detection F1-score confirms that geometric reasoning and projective normalization are effectively handled by the implemented pipeline.

The classification stage, based on SIFT descriptors, Bag-of-Visual-Words encoding and Random Forest inference, achieved moderate performance. While the BoW representation provides a compact fixed-length descriptor, it inherently discards spatial configuration information. This limitation becomes particularly critical in the chess domain, where many pieces share similar local gradient structures and differ mainly in global shape or fine geometric details. As a consequence, the classifier struggles to reliably separate visually similar categories such as king vs queen or bishop vs rook.

A key design objective of this project was to maintain low computational cost and full compatibility with classical computer vision methodologies. The system runs quite efficiently on CPU-only environments, requires no GPU acceleration, and avoids the large memory footprint associated with deep convolutional networks. This makes the solution lightweight, interpretable, and reproducible.

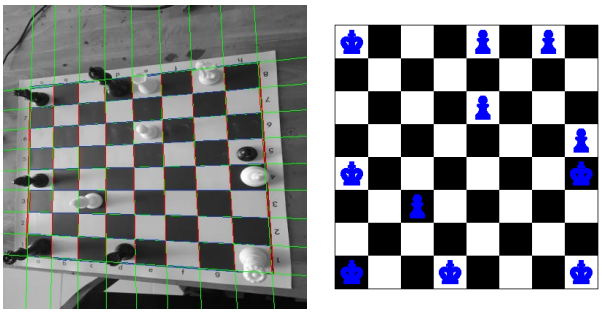
However, these advantages come at the cost of limited semantic discrimination power. Classical local descriptors combined with orderless histogram encoding cannot fully capture complex structural relationships between object parts. Furthermore, the classification performance was severely hindered by the low resolution and overall suboptimal quality of the available training images, which compromised the extraction of robust and discriminative SIFT keypoints. Moreover, sensitivity to illumination variations, partial occlusions between pieces, and small inter-class geometric differences remain open challenges.

Future improvements could explore richer spatial representations, the integration of shape-based geometric features, or hybrid approaches combining handcrafted descriptors with shallow learning architectures. Such extensions could improve discriminative capacity while still maintaining moderate computational requirements.

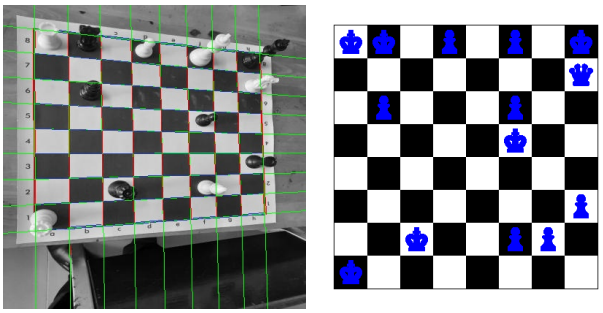
In summary, the proposed system achieves reliable board detection with efficient classical techniques, while highlighting the intrinsic limitations of traditional feature-engineering pipelines when addressing fine-grained object classification tasks.

11 Visual Results (Output Images)

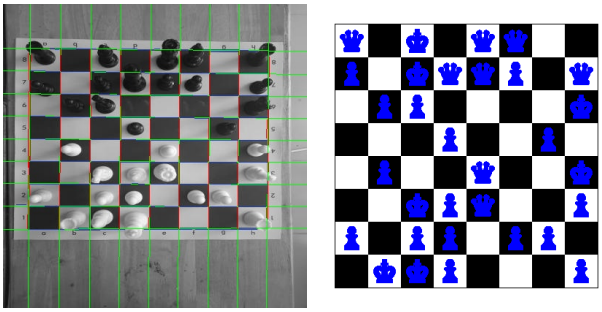
Here we show the visual results for all 55 test images in the dataset 1. Each result is displayed as a pair: the original input image overlaid with the detected grid guidelines, alongside the corresponding 2D virtual chessboard generated by the ChessBoard class. Note that for the 4 samples where grid extraction was incomplete we display only the classification outcomes for the successfully detected cells.



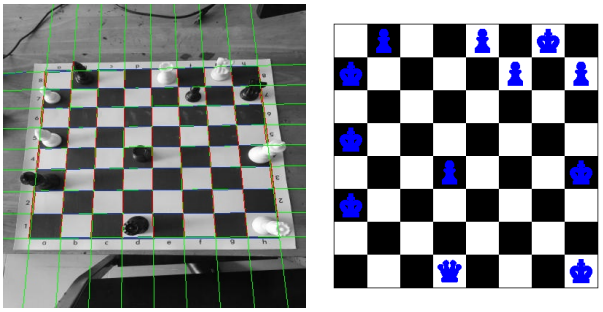
Test image 0



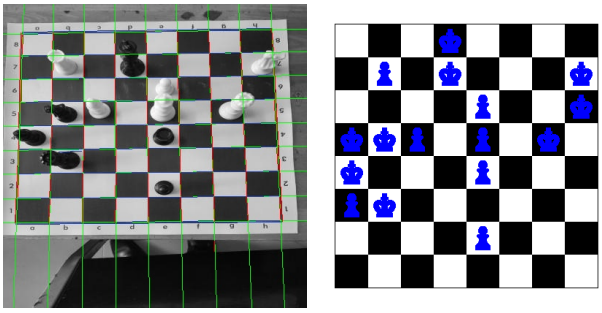
Test image 1



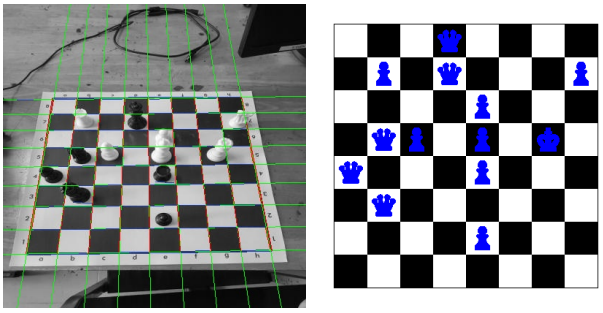
Test image 2



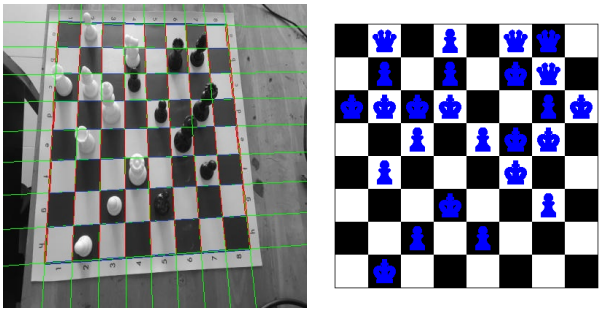
Test image 3



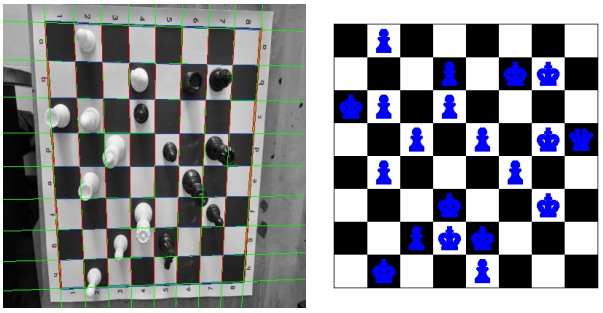
Test image 4



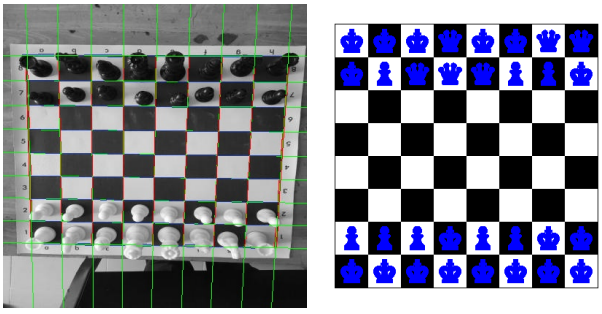
Test image 5



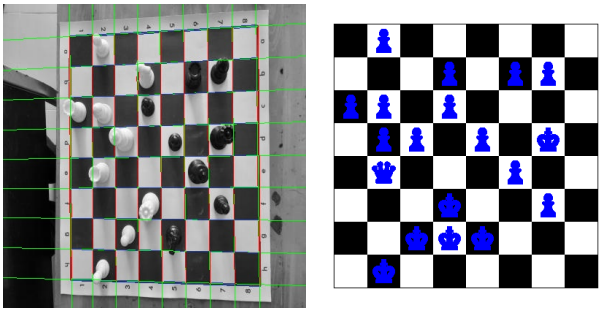
Test image 6



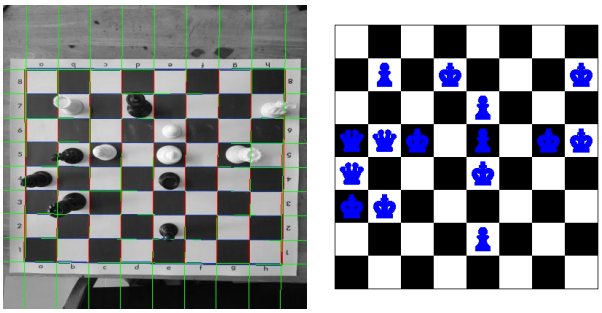
Test image 7



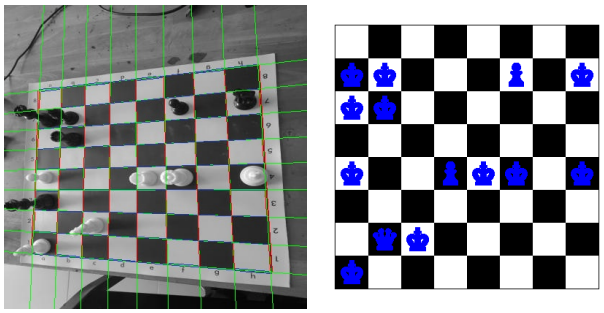
Test image 8



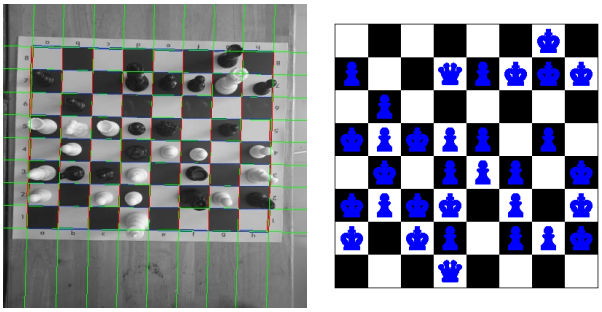
Test image 9



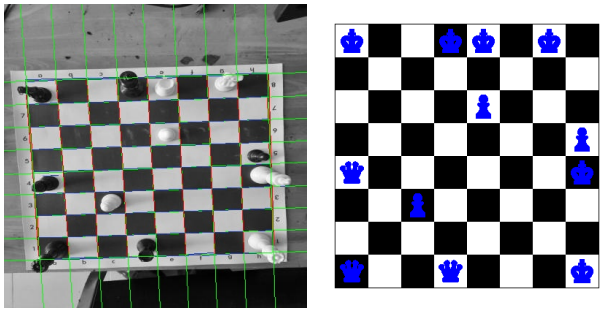
Test image 10



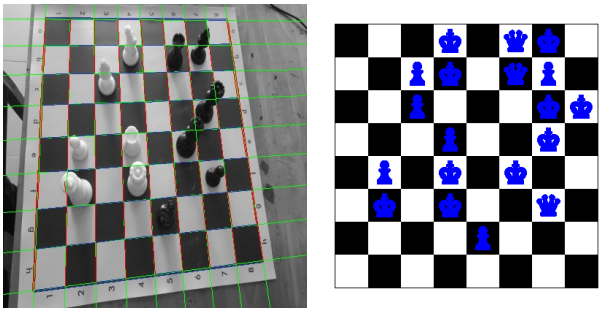
Test image 11



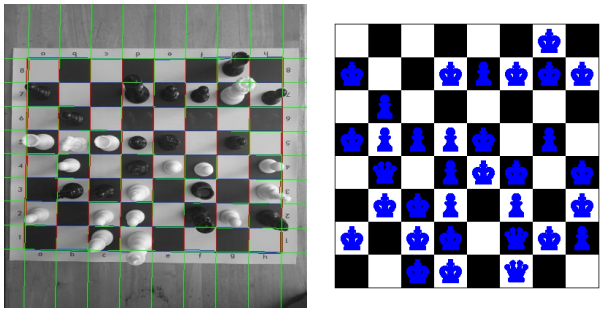
Test image 12



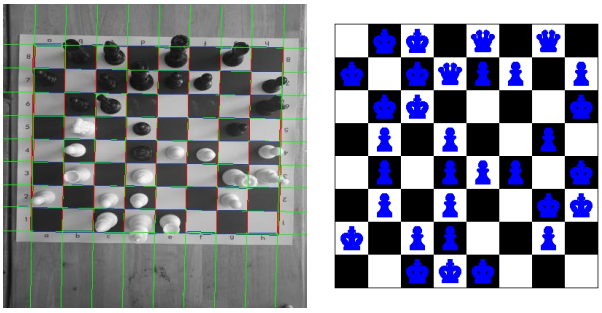
Test image 13



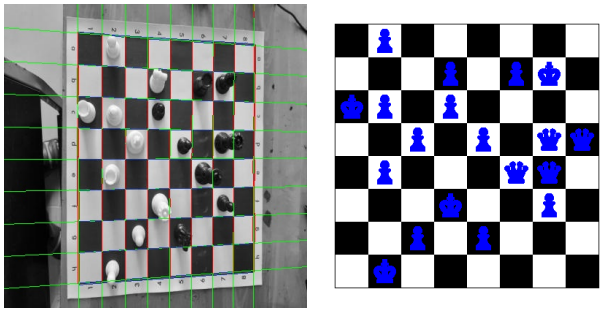
Test image 14



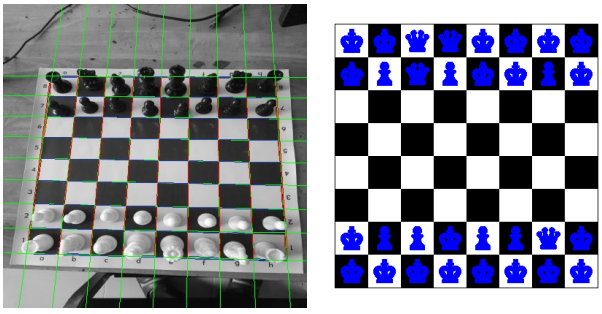
Test image 15



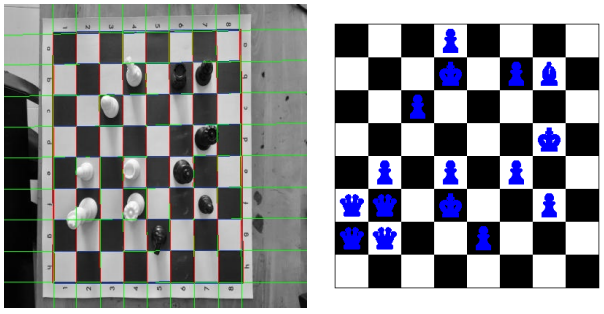
Test image 16



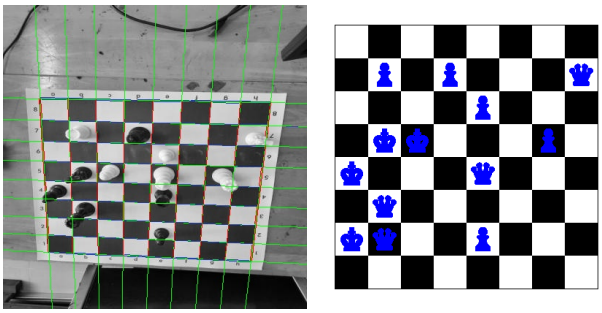
Test image 17



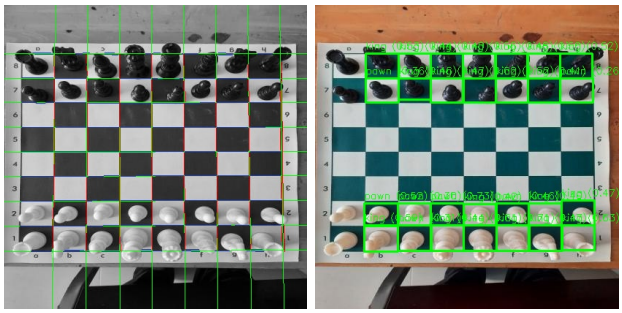
Test image 18



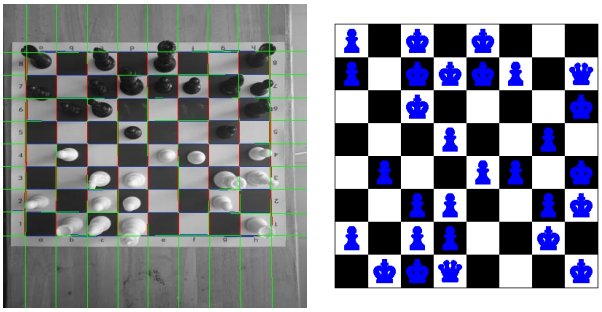
Test image 19



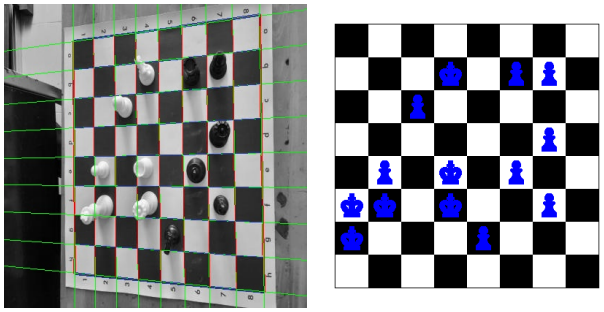
Test image 20



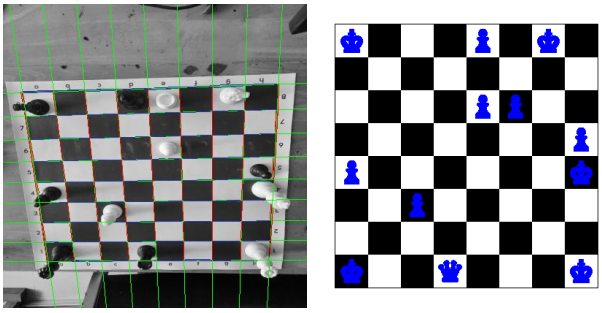
Test image 21



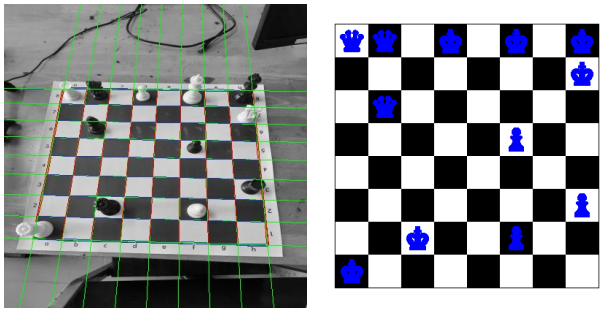
Test image 22



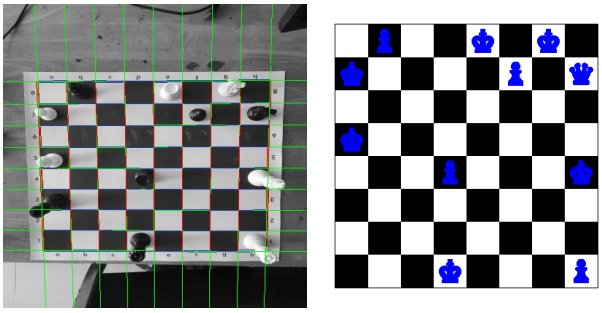
Test image 23



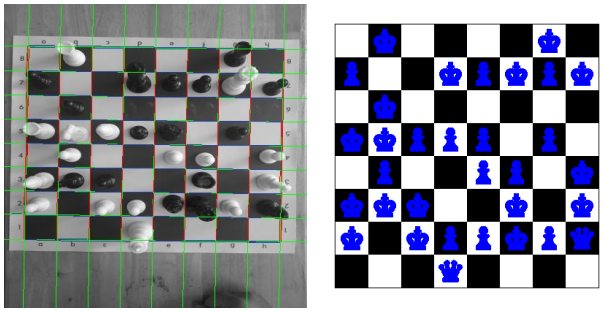
Test image 24



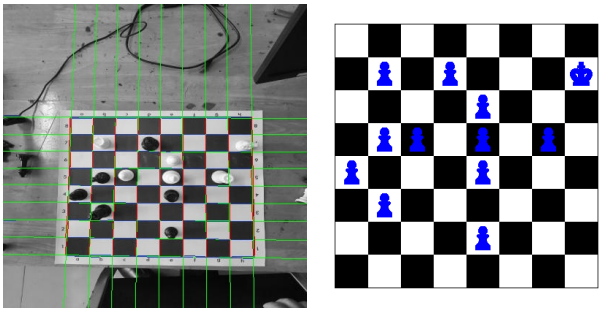
Test image 25



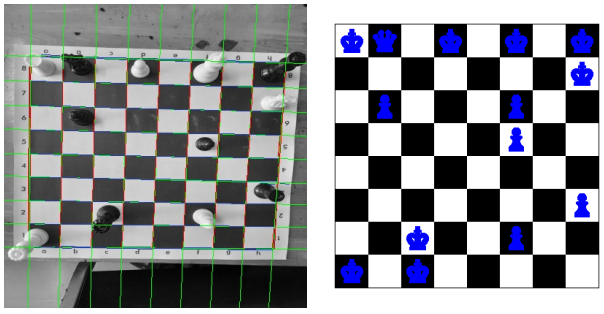
Test image 26



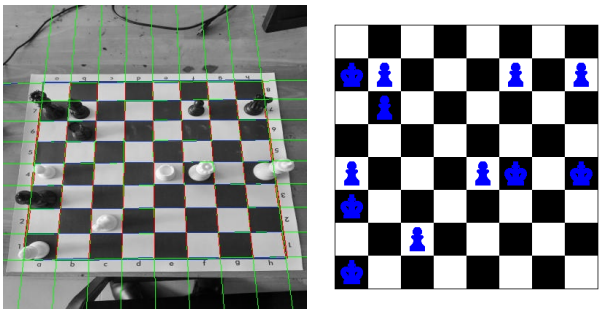
Test image 27



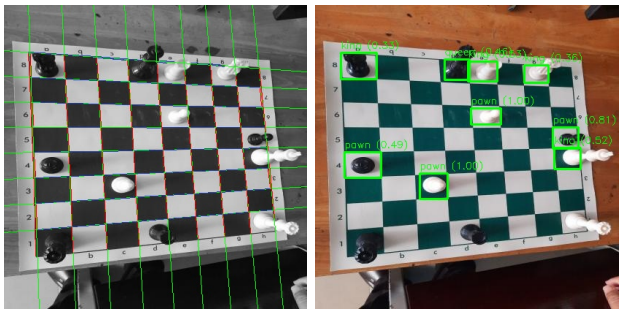
Test image 28



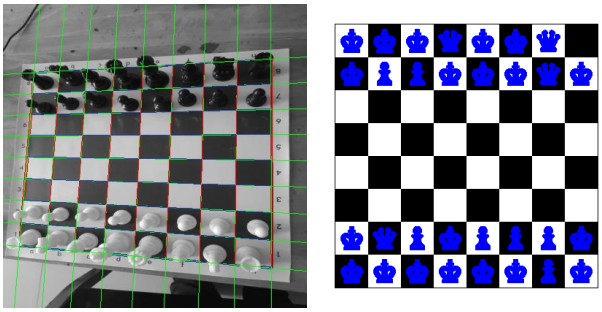
Test image 29



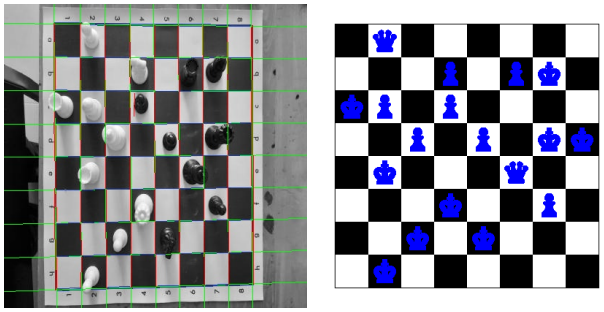
Test image 30



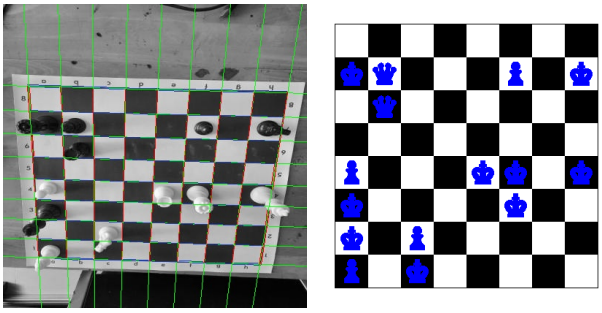
Test image 31



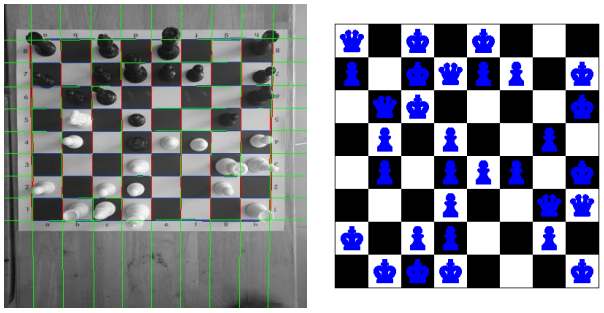
Test image 32



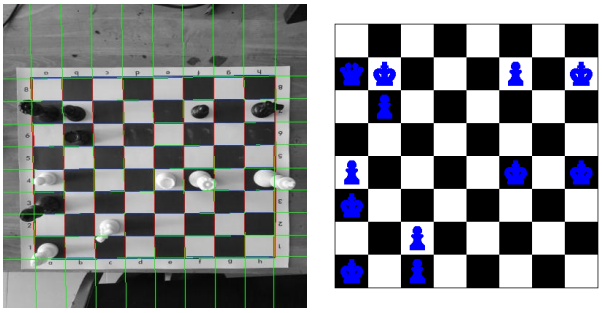
Test image 33



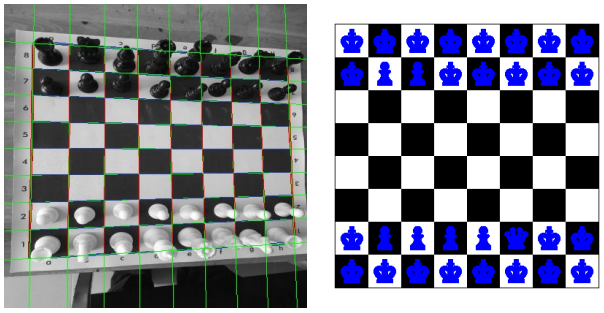
Test image 34



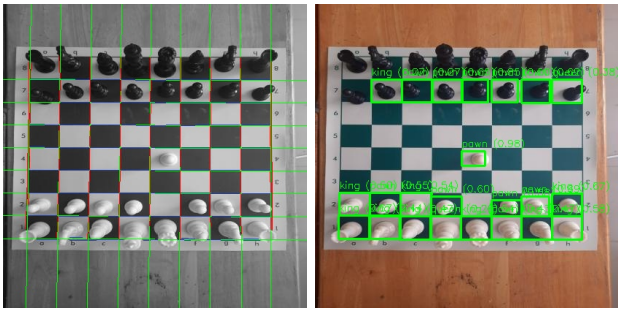
Test image 35



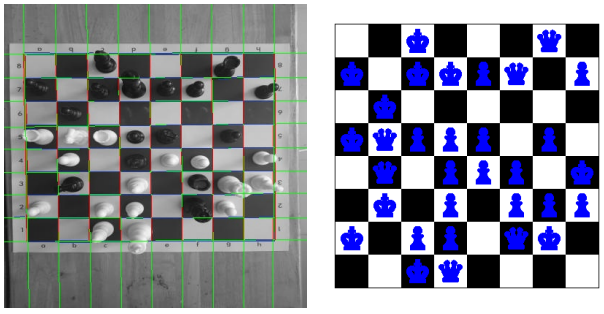
Test image 36



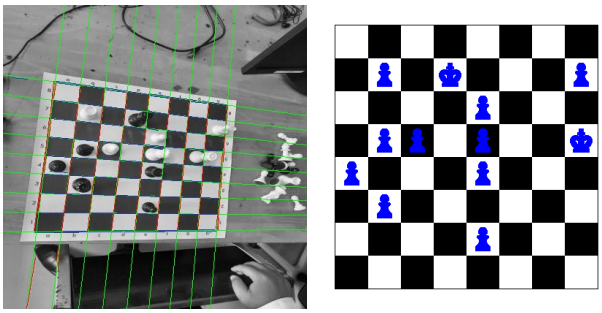
Test image 37



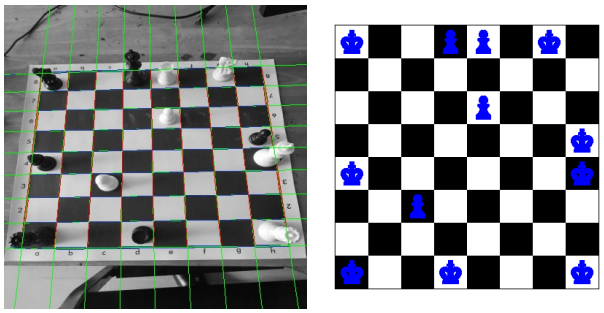
Test image 38



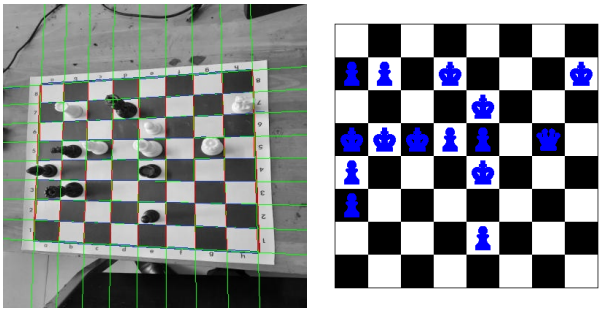
Test image 39



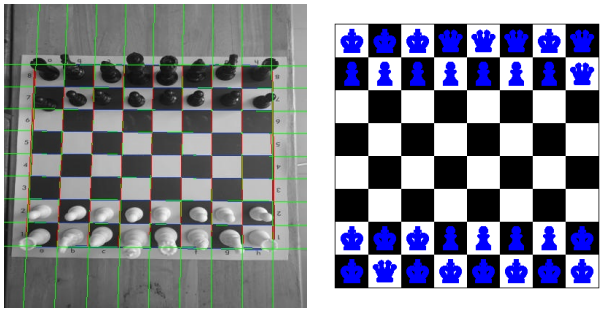
Test image 40



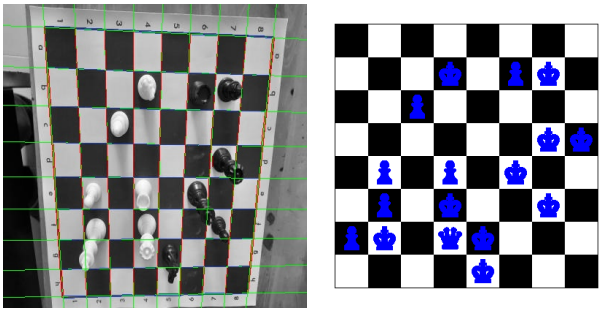
Test image 4



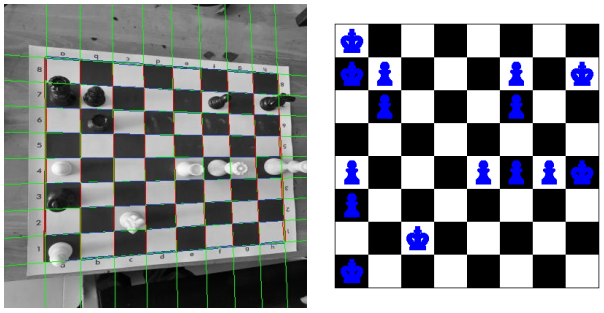
Test image 42



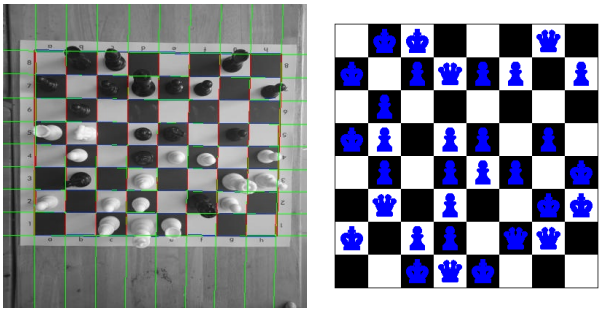
Test image 43



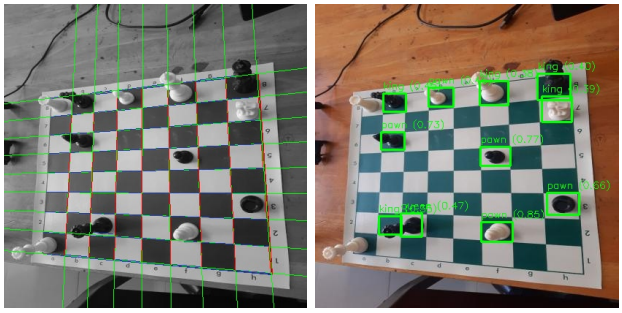
Test image 44



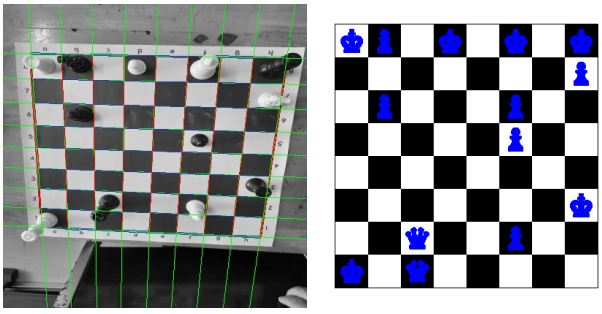
Test image 45



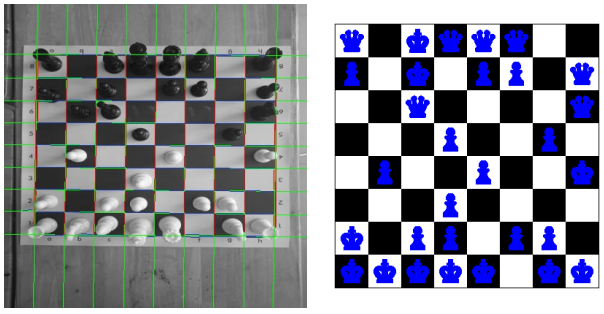
Test image 46



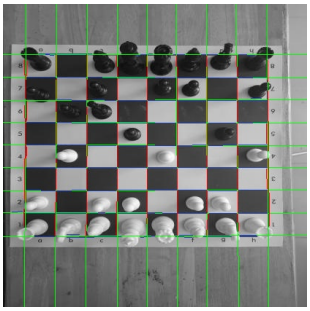
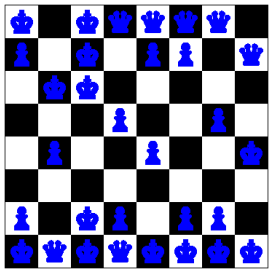
Test image 47



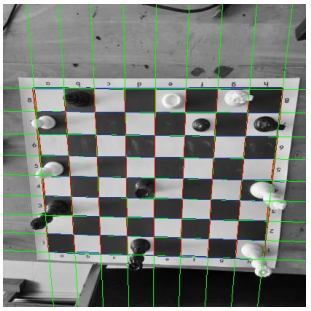
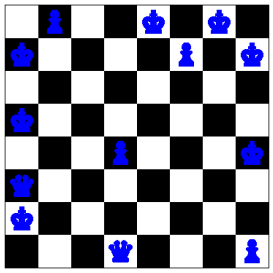
Test image 48



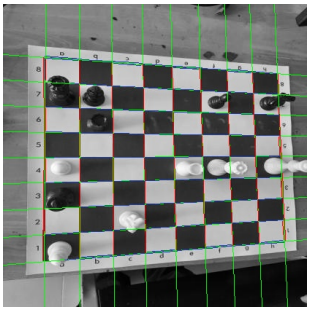
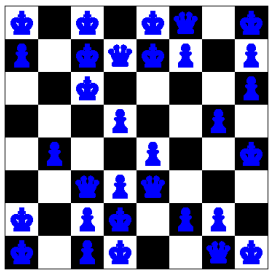
Test image 49

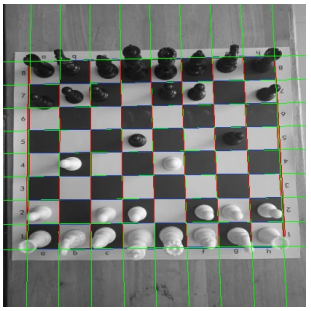
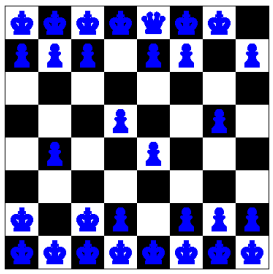
Test image 50

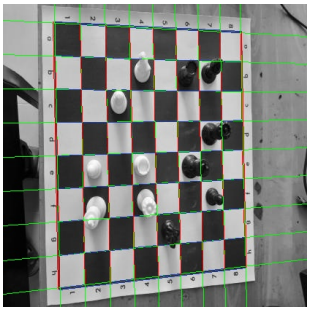
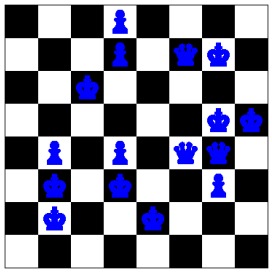
Test image 51

Test image 52

Test image 53

Test image 54